Hardware based Order Book Design in High Frequency Algo Trading

Mohamed Asan Basiri M,
Department of ECE, IIITDM Kurnool, India,
Email - asan@iiitk.ac.in

***Abstract*—The Order Book is an inevitable electronic document used in various business trading and financial sectors to store the track of financial transactions. Since the High Frequency Algo Trading demands fast response, hardware is the best choice as compared with software based designs. This paper proposes the hardware implementation of the Order Book architecture using $45nm$ CMOS technology, where the Order Book details are stored in the hardware Buffer. Also, the same proposed Order Book architecture is implemented with Artix-7 FPGA evaluation board, where the Order Book details are stored in the Block RAM (BRAM).**

***Index Terms*—Block RAM, FPGA, Order Book**

I. Introduction

In todays world, Order Books are used in various business trading and financial sectotors. The literature [1]-[3] show the applications of Order Book in various financial and business sectors. An Order Book is an electronic list of BUY and SELL orders for a specific security or financial instrument, organized by price level. The Order Book information helps traders make better-informed trading decisions. A simple Order Book contains the details as follows. Levels: Several orders at the same price are maintained at the same level. Book depth: The book depth refers simply to the number of levels available at a particular time in the book. Order Book parts: An Order Book contains two parts, namely (a) BUY orders and (b) SELL orders. BUY orders are arranged from the most expensive at the top to the cheapest bid price at the bottom. SELL orders are arranged from the cheapest at the top to the most expensive asking price at the bottom. Table I shows the BUY/SELL Order Book operations.

The literature from [4] to [6] show the operations of Order Book in various financial and business sectors. Very few works on FPGA based Algo Trading [7] are available. In simple, the ADD, DELETE, and MODIFY are the basic three operations performed in the order book. Each Order Book gets the inputs as level, price, quantity, and number of orders. The MODIFY just replaces what's at a particular level. We have assumed in this paper that the price at a level cannot be modified. The quantity and number of orders can be modified. On DELETE, the price/quantity/number of orders from that level gets removed and the levels below are shifted up. In the hardware based order book as mentioned in [8], Cuckoo hash is used to add/delete the entries from/to the order book. Here, the hardware cost is very high due to this hash operation. Many software based designs are available for Order Book implementation. The hardware is always faster than the software. This gives the motivation for our proposed design that can be used in High Frequency Algo Trading that demands the fast response.

TABLE I
Operation of BUY/SELL Order Book

ORDER BOOK: Operation	Level	Price	Quantity	No. of orders
BUY: Initial				
BUY: ADD (level, Price, Quan -tity, No. of orders): -,200,10,5	0	200	10	5
BUY: ADD (level, Price, Quan -tity, No. of orders): -,190,10,10	0 1	200 190	10 10	5 10
BUY: ADD (level, Price, Quan -tity, No. of orders): -,195,1,1	0 1 2	200 195 190	10 1 10	5 1 10
BUY: DELETE (level 0)	0 1	195 190	1 10	1 10
SELL: Initial	0	200	10	5
SELL: ADD (level, Price, Quan -tity, No. of orders): -,215,10,10	0 1	200 215	10 10	5 10
SELL: ADD (level, Price, Quan -tity, No. of orders): -,210,1,1	0 1 2	200 210 215	10 1 10	5 1 10
SELL: MODIFY (level, Price, Quan -tity, No. of orders): 1,210,20,10	0 1 2	200 210 215	10 20 10	5 10 10
SELL: DELETE (level 1)	0 1	200 215	10 10	5 10

The rest of the paper is organized as follows. The hardware based Order Book implementation is shown in Section II. Design modelling, implementations, and results are shown in Section III, followed by a conclusion in Section IV.

II. Hardware based Order Book Implementation

Fig. 1(a) shows the architecture of order book with nine levels. The BUFFER architecture for SELL order book with nine levels is shown in Fig. 1(b). Similar BUFFER is used for BUY order book as well. Table II shows the inputs of Order Book used in Fig. 1(a) that comprises the following attributes.

Both the architectures SELL and BUY Order Books are more similar. Both the architectures SELL and BUY are segregated into control and data paths. Control path includes three Look-up-Tables (LUTs), namely (a) LUT for ADD, (b) LUT for DELETE, and (c) LUT for MODIFY. Inputs to the ADD LUT : input_price and final outputs of the BUFFER from the previous cycle. Input to the DELETE and MODIFY LUTs : input_level. The outputs of the LUTs will be sent to the CONTROL UNIT that generate the nine numbers of 3-bit control signals s0, s1, ...s8. The CONTROL UNIT will be

978-1-7281-8753-2/21/$31.00 ©2021 IEEE
DOI 10.1109/iSES52644.2021.00073

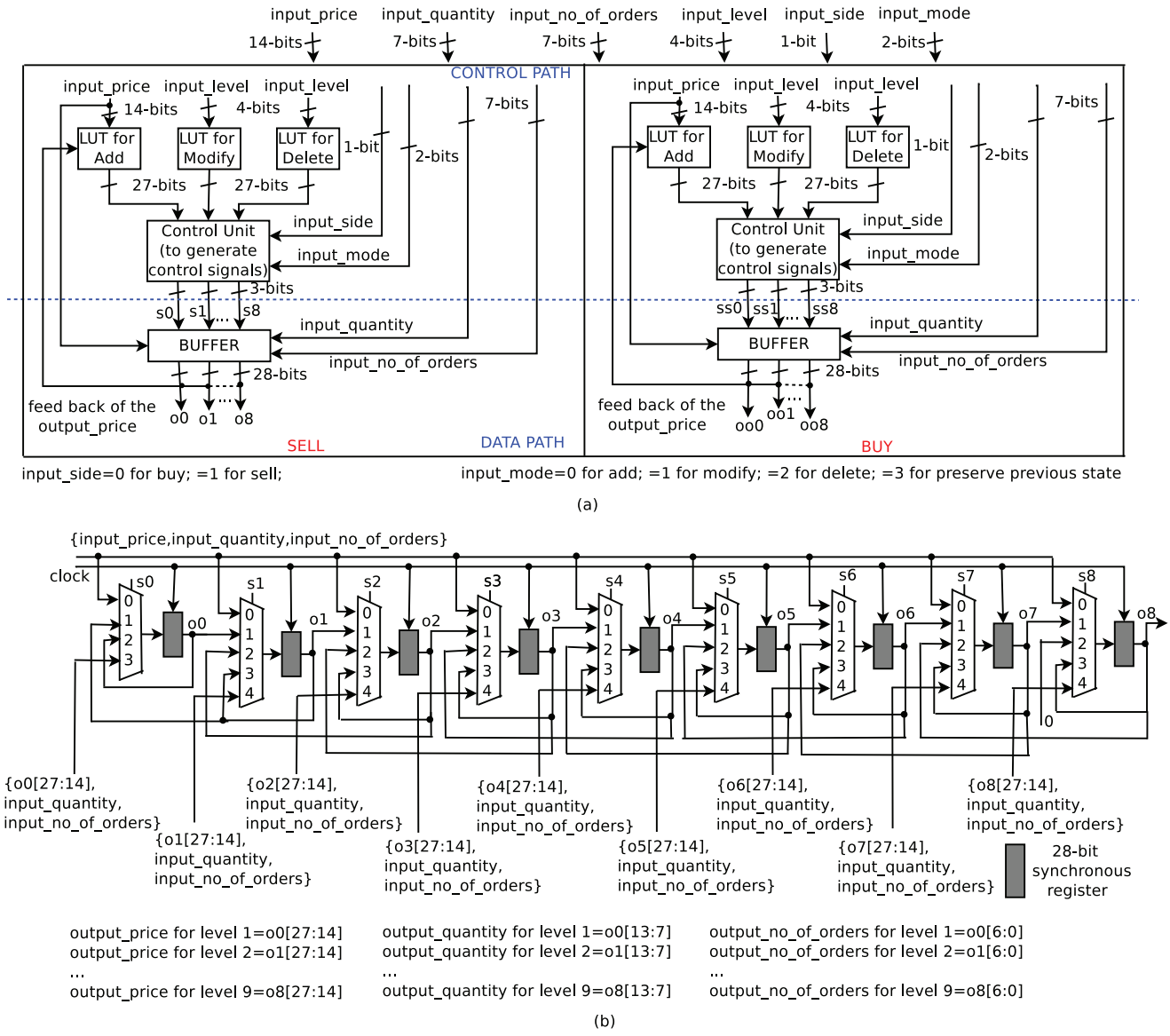

Fig. 1. The proposed (a) architecture of Order Book with nine levels and (b) buffer architecture with nine levels used in SELL order book.

generating the control signals with respect to (a) LUT outputs, (b) current input_side, and (c) current input_mode. These control signals are the select lines of the MULTIPLEXERS in BUFFER. Table III shows the list of control signals generated from the SELL/BUY Order Books as shown in Fig. 1(a). In the BUFFER, nine multiplexers and 28-bit synchronous registers are used. All the registers are connected with a common clock. The operations ADD, MODIFY, DELETE, and PRESERVE can be done in the BUFFER with respect to the select lines of the MULTIPLEXERS. During the PRESERVE operation, the previous state of the BUFFER will be maintained during the current cycle. The outputs of the SELL order book are output_price, output_quantity, and output_no_of_levels. These are obtained from the BUFFER outputs. The SELL order book BUFFER outputs are o0 to o8. Similarly, the BUY order book BUFFER outputs are oo0 to oo8. In Fig. 1(b), the Order Book levels 1 to 9 are corresponding to the outputs o0 to o8 respectively. In other words, the Order Book levels 1 to 9 are corresponding to the multiplexers with select lines s0 to s8 respectively.

The register outputs o0 to o8 are 28-bits wide. Here, the most significant 14-bits ([27:14]) of o0 to o8 represent the output_price associated with the particular level. These output price values (the most significant 14-bits ([27:14]) of o0 to o8) are sent as input to the ADD LUT. Similarly, the least significant 7 bits ([6:0]) of o0 to o8 represent

the output_no_of_orders associated with the particular level. Also, the bit positions [13:7] of o0 to o8 represent the output_quantity associated with the particular level. Tables IV and V show the ADD LUT details of SELL and BUY Order Books as shown in Fig. 1(a) respectively. Similarly, Table VI show the MODIFY and DELETE LUTs used in the SELL/BUY Order Books as shown in Fig. 1(a) respectively. The operations ADD, MODIFY, and DELETE can be done in Fig. 1(a) using the inputs as shown in Table I.

TABLE II
INPUTS OF ORDER BOOKS USED IN FIG. 1(A)

Input	Number of bits used
input_side	1 bit (0 for BUY; 1 for SELL)
input_mode	2 bit (0 for ADD; 1 for MODIFY; 2 for DELETE; 3 for PRESERVE);
input_level	1-9 levels (4-bits)
input_price	14-bits
input_quantity	7-bits
input_no_of_orders	7-bits

TABLE III
CONTROL UNIT DESCRIPTION OF SELL(BUY) ORDER BOOKS AS SHOWN IN FIG. 1(A).

Condition	Control unit outputs (27-bits) {s8,s7,s6,s5,s4,s3,s2,s1,s0}
input_side=0	{3,3,3,3,3,3,3,3,2}
input_side=1(0) && input_mode=0	Output of ADD LUT
input_side=1(0) && input_mode=1	Output of MODIFY LUT
input_side=1(0) && input_mode=2	Output of DELETE LUT
input_side=1(0) && input_mode=3	{3,3,3,3,3,3,3,3,2}

TABLE IV
ADD LUT DESCRIPTION OF SELL ORDER BOOK AS SHOWN IN FIG. 1(A).

Condition	Output (27-bits) {s8,s7,s6,s5,s4,s3,s2,s1,s0}
output_price for level 1 = 0	{1,1,1,1,1,1,1,1,0}
output_price for level 1 > input_price	{1,1,1,1,1,1,1,1,0}
output_price for level 1 ≤ input_price ≤ output_price for level 2	{1,1,1,1,1,1,1,0,2}
output_price for level 2 ≤ input_price ≤ output_price for level 3	{1,1,1,1,1,1,0,3,2}
output_price for level 3 ≤ input_price ≤ output_price for level 4	{1,1,1,1,1,0,3,3,2}
output_price for level 4 ≤ input_price ≤ output_price for level 5	{1,1,1,1,0,3,3,3,2}
output_price for level 5 ≤ input_price ≤ output_price for level 6	{1,1,1,0,3,3,3,3,2}
output_price for level 6 ≤ input_price ≤ output_price for level 7	{1,1,0,3,3,3,3,3,2}
output_price for level 7 ≤ input_price ≤ output_price for level 8	{1,0,3,3,3,3,3,3,2}
output_price for level 8 ≤ input_price ≤ output_price for level 9	{0,3,3,3,3,3,3,3,2}

III. DESIGN MODELLING, IMPLEMENTATIONS, AND RESULTS

All the existing and proposed designs are modelled in Verilog HDL. The synthesis of all these designs is done with

TABLE V
ADD LUT DESCRIPTION OF BUY ORDER BOOK AS SHOWN IN FIG. 1(A).

Condition	Output (27-bits) {s8,s7,s6,s5,s4,s3,s2,s1,s0}
output_price for level 1 = 0	{1,1,1,1,1,1,1,1,0}
output_price for level 1 < input_price	{1,1,1,1,1,1,1,1,0}
output_price for level 1 ≥ input_price ≥ output_price for level 2	{1,1,1,1,1,1,1,0,2}
output_price for level 2 ≥ input_price ≥ output_price for level 3	{1,1,1,1,1,1,0,3,2}
output_price for level 3 ≥ input_price ≥ output_price for level 4	{1,1,1,1,1,0,3,3,2}
output_price for level 4 ≥ input_price ≥ output_price for level 5	{1,1,1,1,0,3,3,3,2}
output_price for level 5 ≥ input_price ≥ output_price for level 6	{1,1,1,0,3,3,3,3,2}
output_price for level 6 ≥ input_price ≥ output_price for level 7	{1,1,0,3,3,3,3,3,2}
output_price for level 7 ≥ input_price ≥ output_price for level 8	{1,0,3,3,3,3,3,3,2}
output_price for level 8 ≥ input_price ≥ output_price for level 9	{0,3,3,3,3,3,3,3,2}

TABLE VI
MODIFY/DELETE LUT DESCRIPTION OF SELL/BUY ORDER BOOKS AS SHOWN IN FIG. 1(A).

	MODIFY	DELETE
input_level	Output (27-bits) {s8,s7,s6,s5,s4,s3,s2,s1,s0}	
1	{3,3,3,3,3,3,3,3,3}	{2,2,2,2,2,2,2,2,1}
2	{3,3,3,3,3,3,3,4,2}	{2,2,2,2,2,2,2,2,2}
3	{3,3,3,3,3,3,4,3,2}	{2,2,2,2,2,2,2,3,2}
4	{3,3,3,3,3,4,3,3,2}	{2,2,2,2,2,2,3,3,2}
5	{3,3,3,3,4,3,3,3,2}	{2,2,2,2,2,3,3,3,2}
6	{3,3,3,4,3,3,3,3,2}	{2,2,2,2,3,3,3,3,2}
7	{3,3,4,3,3,3,3,3,2}	{2,2,2,3,3,3,3,3,2}
8	{3,4,3,3,3,3,3,3,2}	{2,2,3,3,3,3,3,3,2}
9	{4,3,3,3,3,3,3,3,2}	{2,3,3,3,3,3,3,3,2}

Cadence Genus and Innovus 15.20 using $45\ nm$ CMOS technology, where the operating voltage is $0.9v$. Table VII shows the comparison of critical path delay, total area, PDP, and throughput between the various proposed designs of entropy calculation. Here, Power-delay-product (PDP) or energy per operation = (switching power + leakage power) $\times$ critical path delay.

TABLE VII
SYNTHESIS RESULTS OF ORDER BOOK (WITH NINE LEVELS AS SHOWN IN FIG 1(A) WITH SPECIFICATIONS AS SHOWN IN TABLE II) USING $45nm$ CMOS TECHNOLOGY WITH CADENCE.

Critical path delay (ps)	000685.00	Switching Power (nw)	137414.91
Frequency (MHz)	001459.85	Leakage Power (nw)	000492.99
Number of Cells	003350.00	PDP ($\times 10^{-19} J$)	944669.11
Total Area (μm^2)	009421.76		

Fig. 2 shows the complete block diagram of 32-bit Microblaze, custom IP (SELL Order Book Co-processor with five levels for ADD operation), BRAM to store the Order Book Contents from the Co-processor, and other peripherals with 32-bit AXI_LITE bus interconnect using Xilinx Vivado. Here, the peripheral AXI timer is used to measure the number of cycles

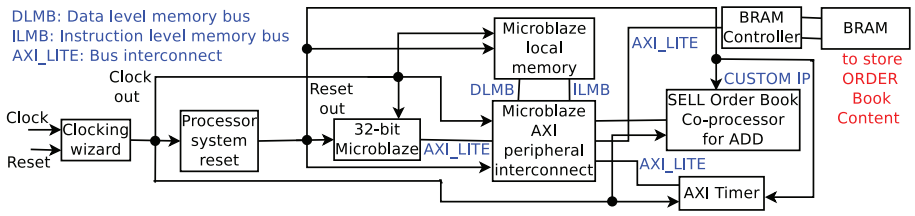

Fig. 2. Block diagram of 32-bit Microblaze, custom IP (SELL Order Book Co-processor with five levels for ADD operation), BRAM to store the Order Book Contents from the Co-processor, and other peripherals with AXI_LITE using Xilinx Vivado.

TABLE VIII
SYNTHESIS RESULTS OF SELL ORDER BOOK HARDWARE IMPLEMENTATION WITH FIVE LEVELS FOR ADD OPERATION USING ARTIX-7 NEXYS-4 FPGA (XC7A100T-CSG324) WITH XILINX VIVADO ($100MHz$)

	No. of cycles	Utilization			Power (w)	
	ADD operation	No. of Slices ($\leq$ 15850)	No. of LUTs as logic ($\leq$ 63400)	No. of BRAM tile ($\leq$ 135)	Switching power	Leakage power
Proposed with a co-processor (Fig. 2)^h	54	1118	2172	18	0.040	0.098
Proposed without a co-processor in Fig. 2^s	11562	372	879	12	0.018	0.098
Cuckoo hash based [8]^h	63	1828	3100	18	0.052	0.098

h and s are hardware and software based implementations respectively.

required for the operations involved in the software (32-bit Microblaze). In this hardware-software co-design, the software and hardware portions are implemented with c and $verilog$ HDL respectively. The software and hardware portions are run by 32-bit Microblaze and Co-processor respectively. The co-processor is to perform the ADD operation for the SELL Order Book. Here, the co-processor will get the inputs from the main processor (32-bit Microblaze) and results (quantity, number of orders, price, and level) from the co-processor are stored in BRAM then the results will be sent to the main processor. A cycle is required for a 32-bit data transmission between the Microblaze & co-processor and between Microblaze and BRAM. Table VIII shows the synthesis results of of SELL Order Book implementation with five levels for ADD operation with and without the co-processor using Artix-7 Nexys-4 FPGA (XC7A100T-CSG324) with Xilinx Vivado ($100MHz$), where 54 cycles are required for the ADD operation. Tables VIII clearly shows that the number of cycles required for the hardware implementation is 99.0% less than the software implementation of the ADD operation of the SELL order book with five levels. Also, the trade-off with the hardware implementation of SELL order book are no. of slices, no. of LUTs, and switching power dissipation. Since the order book values are stored in BRAM of both the hardware and software based implementations as shown in Table VIII, the number of BRAM tile in both these cases is the same.

Buffer. The later case is implemented with Artix-7 FPGA evaluation board, where the Order Book details are stored in the Block RAM.

IV. CONCLUSION

In the modern business and financial organizations use the Order Book as an inevitable electronic document to store the transactions. This paper proposes ASIC and FPGA based hardware implementations of the Order Book design. The former case is implemented using $45nm$ CMOS technology, where the Order Book details are stored in the hardware

REFERENCES

[1] Peter A. Whigham, Rasika Withanawasam, Timothy Crack, and I.M. Premachandra, "*Evolving Trading Strategies for a Limit-order Book Generator*", IEEE Congress on Evolutionary Computation, pp. 1-8, July 2010.

[2] G.C. Silaghi and V. Robu, "*Exploiting the Order Book for Mass Customized Manufacturing Control Systems With Capacity Limitations*", IEEE Transactions on Engineering Management, vol. 54, no. 1, pp. 145-155, Feb. 2017.

[3] Alessio Emanuele Biondo, Alessandro Pluchino, and Andrea Rapisarda, "*Order book, financial markets, and self-organized criticality*", Chaos, Solitons and Fractals Nonlinear Science, and Nonequilibrium and Complex Phenomena, Elsevier, vol. 88, pp. 196-208, Mar. 2016.

[4] Nima Akbarzadeh, Cem Tekin, and Mihaela van der Schaar, "*Online Learning in Limit Order Book Trade Execution*", IEEE Journal of Selected Topics in Signal Processing, vol. 66, no. 17, pp. 4626-4641, Sep. 2018.

[5] D. D. Imaev and D. H. Imaev, "*Automated Trading Systems Based on Order Book Imbalance*", IEEE International Conference on Soft Computing and Measurements (SCM), vol. 66, no. 17, pp. 815-819, May 2017.

[6] Efstathios Panayi, Gareth W. Peters, Jon Danielsson, and Jean-Pierre Zigrand, "*Designating market maker behaviour in limit order book markets*", Econometrics and Statistics, Elsevier, vol. 5, pp. 20-44, Nov. 2016.

[7] Christian Leber, Benjamin Geib, and Heiner Litz, "*High Frequency Trading Acceleration using FPGAs*", 21st IEEE International Conference on Field Programmable Logic and Applications, pp. 317-322, Sep. 2011.

[8] Milan Dvorak and Jan Korenek, "*Low Latency Book Handling in FPGA for High Frequency Trading*", 17th IEEE International Symposium on Design and Diagnostics of Electronic Circuits & Systems, pp. 1-4, April 2014.